

Membrane Gas Separation Simulation with OpenFOAM: A Short Practical Course

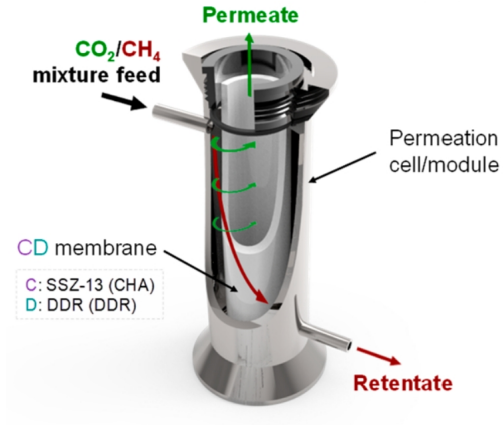
Lecture 00

Professor Adriano Possebon Rosa (aprosa@unb.br)

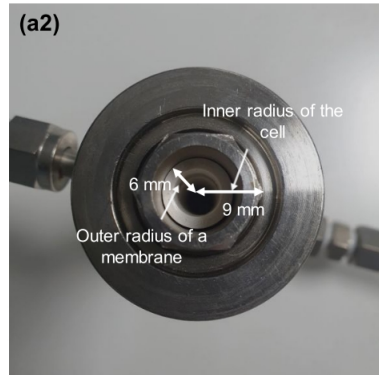
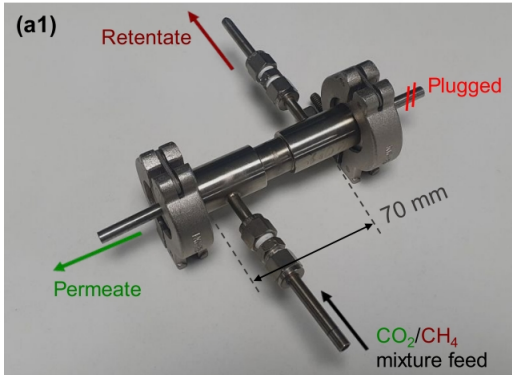
Energy and Environment Laboratory (lea.unb.br)

University of Brasilia

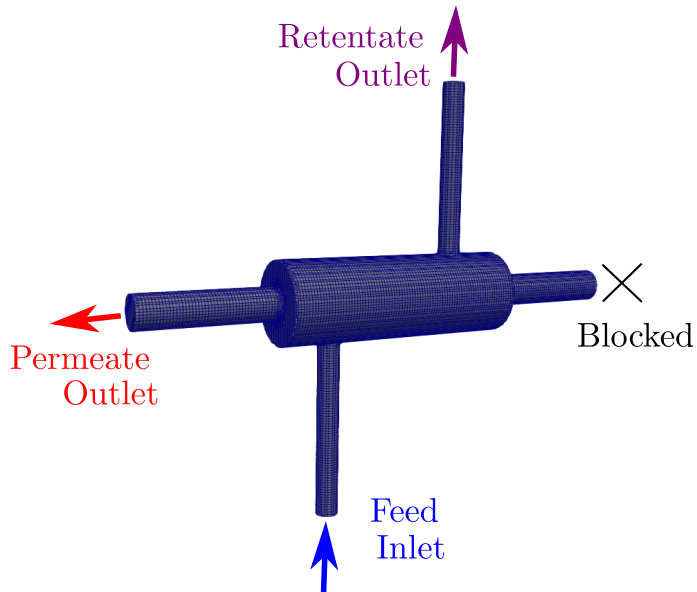
Goal: simulate a membrane gas separation module using Computational Fluid Dynamics (CFD).

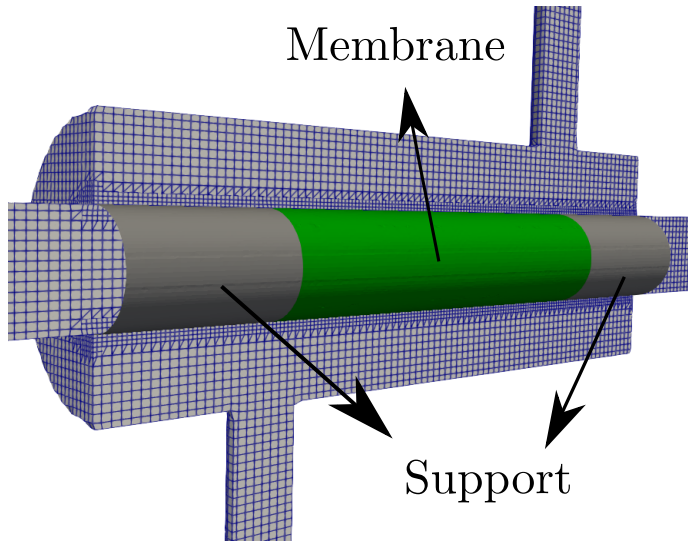


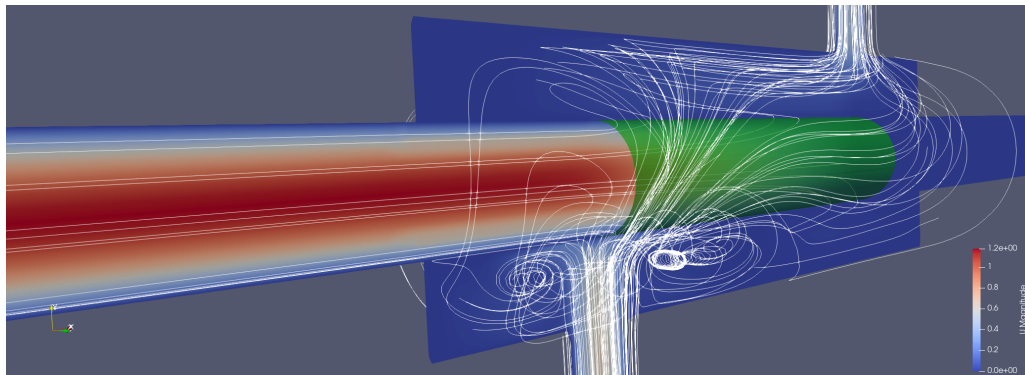
Jeong et al. (2022).

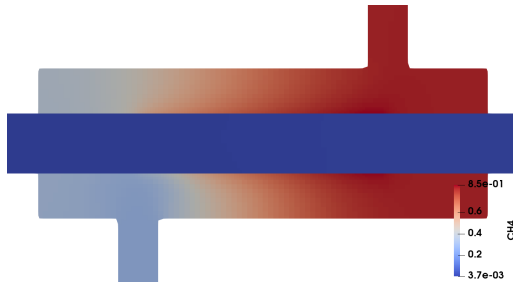


Jeong et al. (2022).









Lectures:

- **Lecture 00:** Membrane Gas Separation; Governing Equations; CFD Basics; Finite Volume Method; OpenFOAM; Simulation: Lid-Driven Cavity.
- **Lecture 01:** Governing Equations (Basic Problems); Simulation: Parallel Plates; Mesh Basics; Mesh Generation with Gmsh; Simulation: T-Junction; Baffles.
- **Lecture 02:** Gas Flow Simulation; Baffles (Membranes); Simulation: T-Junction with Membrane; SnappyHexMesh; Simulation: 3D T-Junction with Membrane and SHM.
- **Lecture 03:** Running the Complete Simulation (Membrane Module); Simulation Setup; Boundary Conditions; Initial Conditions; Simulation and Experiment; Parallelization; Post Process.

Table of Contents

- 1 Membrane Gas Separation
- 2 CFD
- 3 OpenFOAM
- 4 Simulation: Lid-Driven Cavity
- 5 Simulation: 3D Lid-Driven Cavity
- 6 Homework
- 7 Conclusion
- 8 Supplementary Material
- 9 Papers

Gas Separation Process

- Applications:

- Natural and biogas gas processing (CO_2 , H_2S , N_2 separation)
- Air separation (O_2 , N_2 production)
- Hydrogen purification (refineries, fuel cells)
- Carbon capture (flue gas treatment)

- Industry Drivers:

- Energy efficiency demands
- Environmental regulations
- Emerging hydrogen economy

Gas Separation Methods:

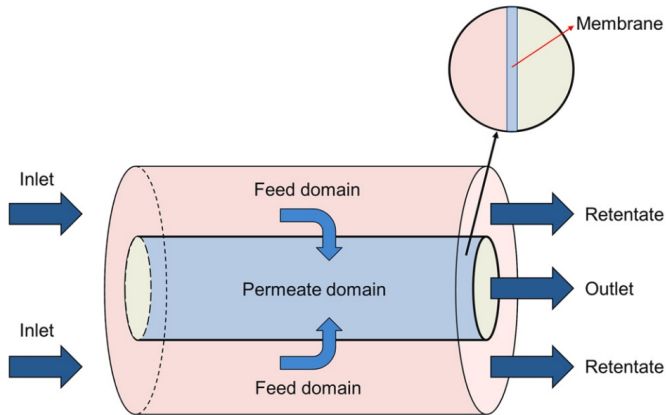
- Cryogenic Distillation: Phase separation via low-temperature condensation
- Absorption: Selective dissolution in liquid solvents
- Adsorption: Surface capture on porous solid substrates
- **Membrane Separation:** Selective permeation through semi-permeable barriers

A **membrane** is a **semi-permeable barrier** that **selectively** permits transport of specific components.

Selectivity: Separates based on size, solubility, or affinity.

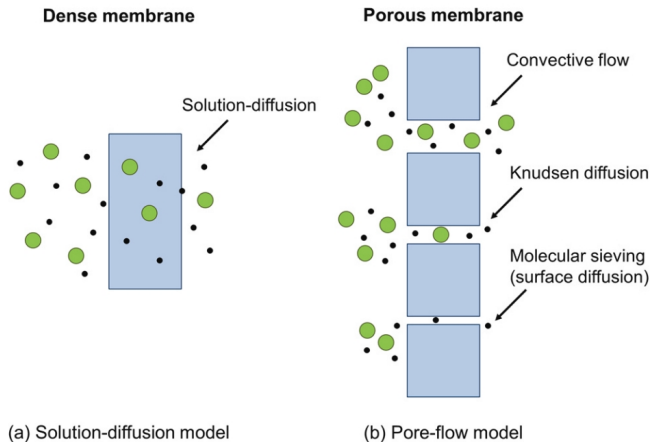
Driving Force: Requires pressure or concentration gradients.

Schematic of general gas separation process in a membrane module.



Foo et al., 2023.

Mechanisms of gas separation through dense and porous membranes.



Foo et al., 2023.

Membranes are employed in:

- the separation of nitrogen and oxygen from air
- removal of water vapor, CO₂, H₂S and N₂ from natural gas
- separation of methane from the other components of biogas
- recovery of hydrogen in oil refinery processes
- enrichment of air by oxygen for medical or metallurgical purposes
- . . .

Ismael et al., 2015.

The Gas Separation process using membranes depend upon:

- Membrane structure and thickness (permeance, separation factors)
- **Membrane configuration** (flat, tubular, hollow fiber, spiral)
- **Module and system design**

Advantages of membranes:

- Continuous operation without phase changes or chemical additives
- Simple in concept and operation
- Modular design enabling flexible scaling
- Lower energy requirements compared to thermal processes
- Equipment size may be decreased

Simple rules for gas separation by membranes to be favorable:

- When moderate purity recovery is sufficient
- When the components to be separated are a considerable amount
- When the feed gas is available at the necessary pressure, or when the residue stream is needed at high pressure
- When the feed gas contains no substances harmful to the membrane
- When a membrane with sufficient selectivity is available

Ismael et al., 2015.

Membrane Material Types

- Polymeric Membranes:
 - Amorphous or glassy polymer matrices
 - Solution-diffusion transport mechanism
- Inorganic Membranes:
 - Zeolites, Perovskites, Carbon nanotubes
 - Molecular sieving via rigid pore structures
- Mixed-Matrix Membranes:
 - Hybrid organic-inorganic composites
 - Synergistic performance enhancement

Zeolites

Crystalline aluminosilicate made up of a $3D$ framework that forms uniformly sized pores of molecular dimensions

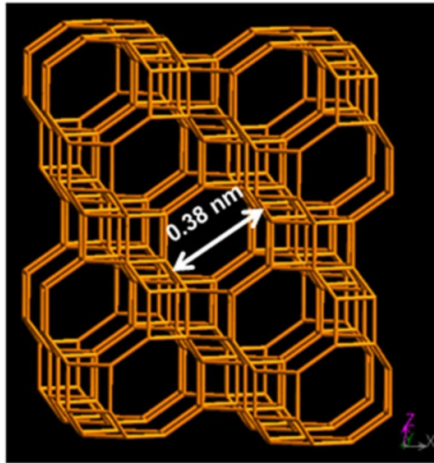
Gas molecules with different sizes and shapes can be separated by zeolites through their channels.

- Advantages

- High selectivity: Size/shape exclusion
- Thermal stability
- Chemical resistance

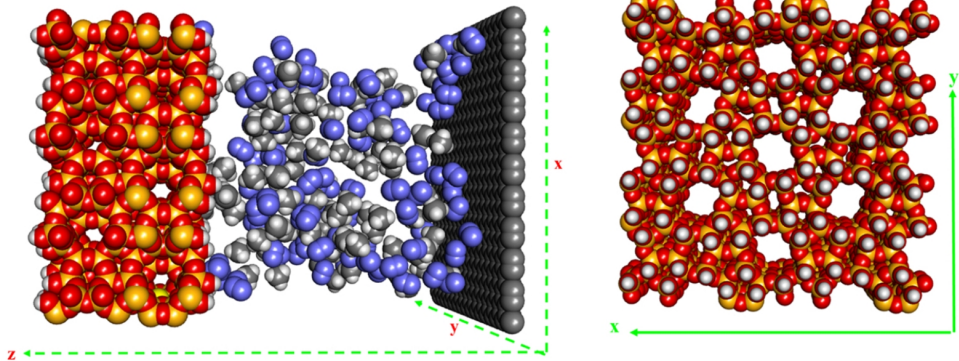
- Challenges

- Brittle - Difficult to scale up
- Defect-sensitive (pinholes reduce selectivity)
- High manufacturing cost



Zeolite SAPO-34 pore structure.

Carreon, 2018.



Molecular Dynamics. NAMD. MFI zeolite, CH₄, N₂ and a graphene sheet.

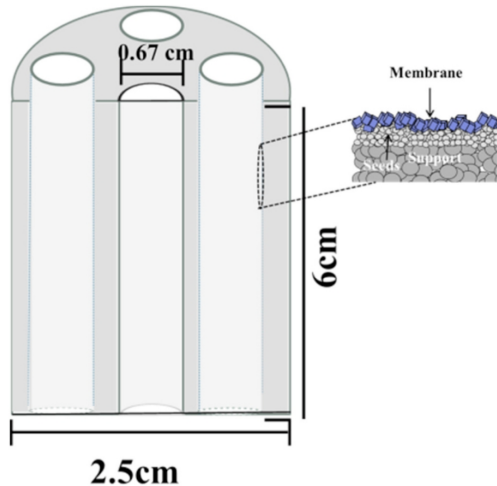
Azizi et al., 2022.

Table 1. Most explored small-pore zeolites [12], their pore architectures and sizes for corresponding pure silica forms [4] and membrane fabrication status.

IZA Code	Zeolite Names	Pore Architecture	Pore Size, Å	Membranes
AEI	SSZ-39, AIPO-18, SIZ-8	$8 \times 8 \times 8$	3.8×3.8	Y
AFX	SAPO-56, SSZ-16	$8 \times 8 \times 8$	3.4×3.6	N
ANA	Analcime, AIPO-24	$8 \times 8 \times 8$	1.6×4.2	Y
CHA	SSZ-13, SAPO-34	$8 \times 8 \times 8$	3.8×3.8	Y
DDR	ZSM-58, Deca-dodecasil 3R	8×8	3.6×4.4	Y
ERI	UZM-12, AIPO-17	$8 \times 8 \times 8$	3.6×5.1	Y
GIS	Gismondine, Zeolite P	$8 \times 8 \times 8$	$3.1 \times 4.5, 2.8 \times 4.8$	Y
IHW	ITQ-32	8×8	3.5×4.3	N
ITE	ITQ-3	8×8	$3.8 \times 4.3, 2.7 \times 5.8$	N
ITW	ITQ-12	8×8	$2.4 \times 5.4, 3.9 \times 4.2$	N
KFI	ZK-5	$8 \times 8 \times 8$	3.9×3.9	N
LEV	Levyne, SAPO-35	8×8	3.6×4.8	N
LTA	Linde Type A, ITQ-29	$8 \times 8 \times 8$	4.1×4.1	Y
NSI	Nu-6(2)	8×8	$2.6 \times 4.5, 2.4 \times 4.8$	N
RHO	RHO	$8 \times 8 \times 8$	3.6×3.6	Y
RTE	RUB-3	8	3.7×4.4	N
RTH	RUB-13, SSZ-50	8×8	$3.8 \times 4.1, 2.5 \times 5.6$	N
RWR	RUB-24	8×8	2.8×5.0	N
SAS	STA-6, SSZ-73	8	4.2×4.2	N
SAV	STA-7	$8 \times 8 \times 8$	$3.8 \times 3.8, 3.9 \times 3.9$	N
UFI	UZM-5	8×8	$3.6 \times 4.4, 3.2 \times 3.2$	N

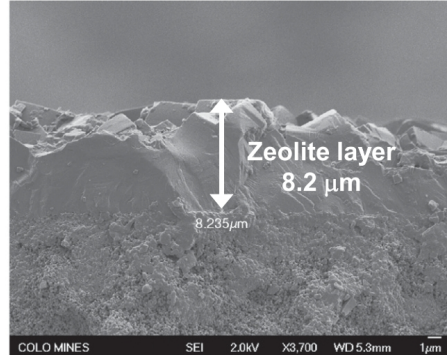
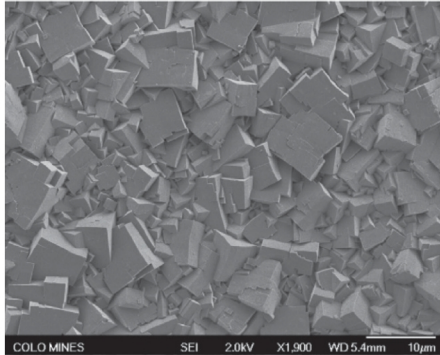
Cao et al., 2022.

Gas	Kinematic Diameter (nm)
CH ₄	0.380
N ₂	0.364
O ₂	0.346
CO ₂	0.330
H ₂	0.289

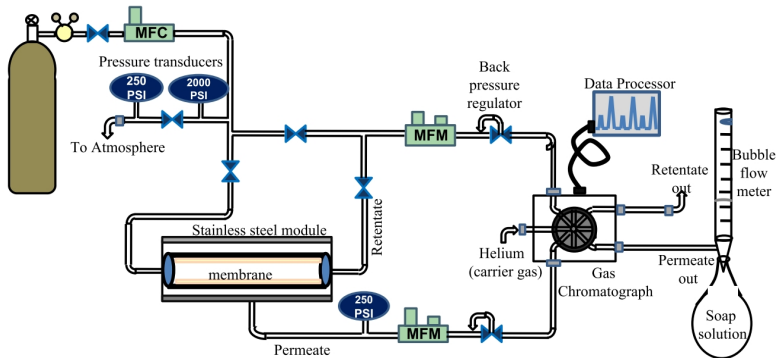


Schematic diagram of 7-channel SSZ-13 monolith membrane.

Li et al., 2021.



Li et al., 2015.



Gas permeation testing system.

Tubes: 6 *cm* long. Area $\approx 7 \text{ cm}^2$.

Feed: 50/50 N_2/CH_4 , 100 *mL/min*.

Trans-membrane pressure drop: 100 – 500 *kPa*.

Li et al., 2015.

Membrane Performance Metrics

Permeance:

$$P_{m,i} = \frac{Q_i}{A_m \cdot t \cdot \Delta P_i}$$

Flux normalized by the driving force.

Speed at which molecules permeate the membrane.

Q_i is the moles of component i permeated over a time period t .

ΔP_i is the partial pressure difference across the membrane.

A_m is the membrane surface area.

Permeance Units

- SI Units:

$$\frac{\text{mol}}{\text{m}^2 \cdot \text{s} \cdot \text{Pa}}$$

- Gas Permeance Unit (GPU):

$$1 \text{ GPU} = 3.35 \times 10^{-10} \frac{\text{mol}}{\text{m}^2 \cdot \text{s} \cdot \text{Pa}}$$

Ideal selectivity:

$$\alpha_{i/j}^0 = \frac{P_{m,i}^0}{P_{m,j}^0}$$

Ratio of permeances for two different components, i and j , measured independently with pure feed.

Separation selectivity:

$$\alpha_{i/j} = \frac{y_i/y_j}{x_i/x_j}$$

Separation selectivity from a mixture separation test.

x and y are molar fractions of the feed and permeate gas.

Permeability:

$$P_{b,i} = P_{m,i}\delta$$

Permeance multiplied by membrane thickness.

Example from Cao et al., 2022.

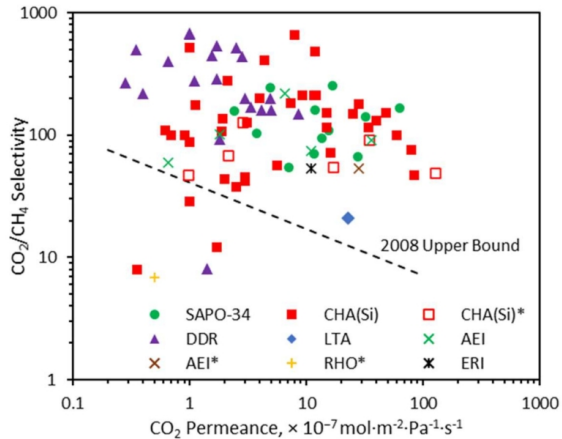
CH₄ / N₂

SSZ-13 zeolite membrane. $\Delta P = 0.303$ MPa

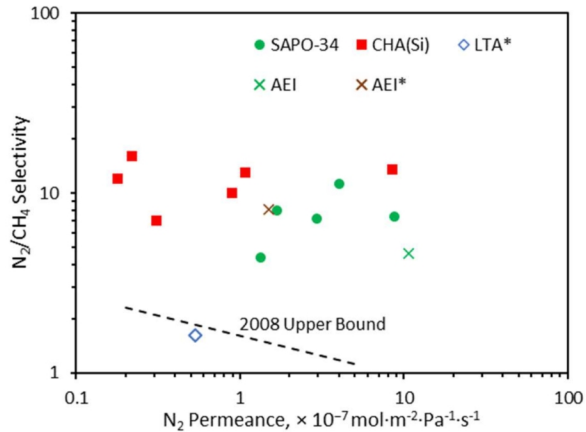
$$P_{N_2} = 8.5 \times 10^{-7} \frac{\text{mol}}{\text{m}^2 \cdot \text{s} \cdot \text{Pa}}$$

$$P_{CH_4} = 6.3 \times 10^{-8} \frac{\text{mol}}{\text{m}^2 \cdot \text{s} \cdot \text{Pa}}$$

$$\alpha_{N_2/CH_4} = 13.5$$



Cao et al., 2022.



Cao et al., 2022.

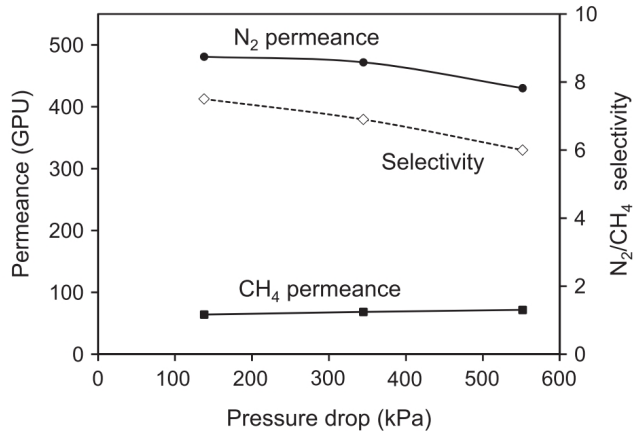
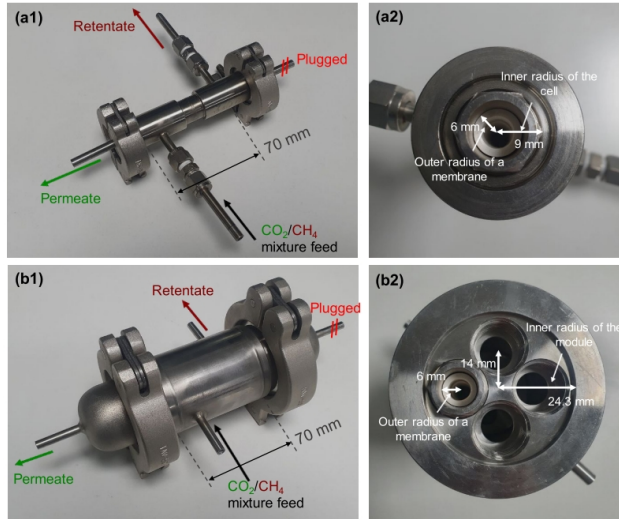


Fig. 15. Permeances and selectivity of a N₂/CH₄ mixture (50/50) at 23 °C as a function of trans-membrane pressure drop for SAPO-34 membrane M-100nm-6h-1. The permeate pressure was kept at 85 kPa.

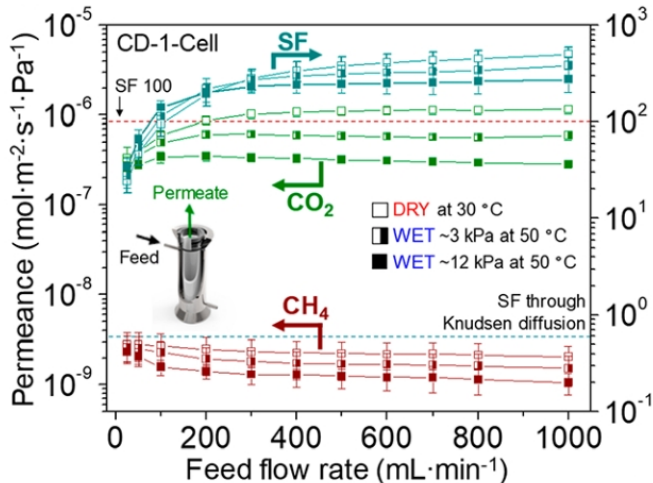
Li et al., 2015.

Experiment to compare.



Jeong et al., 2022.

Permeance.

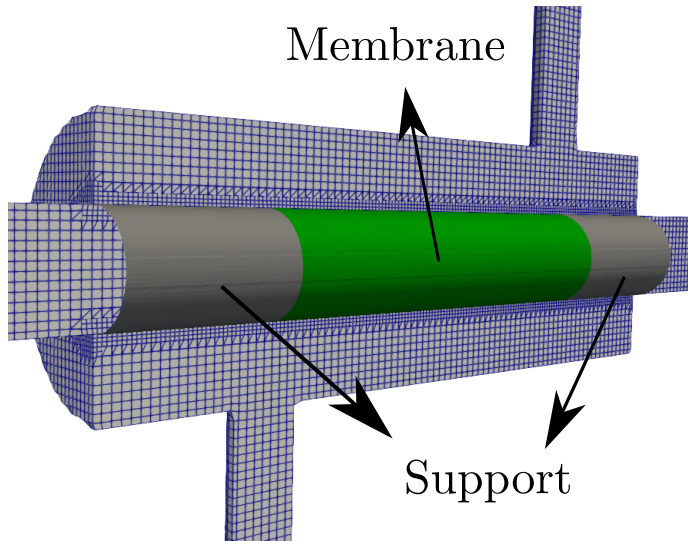


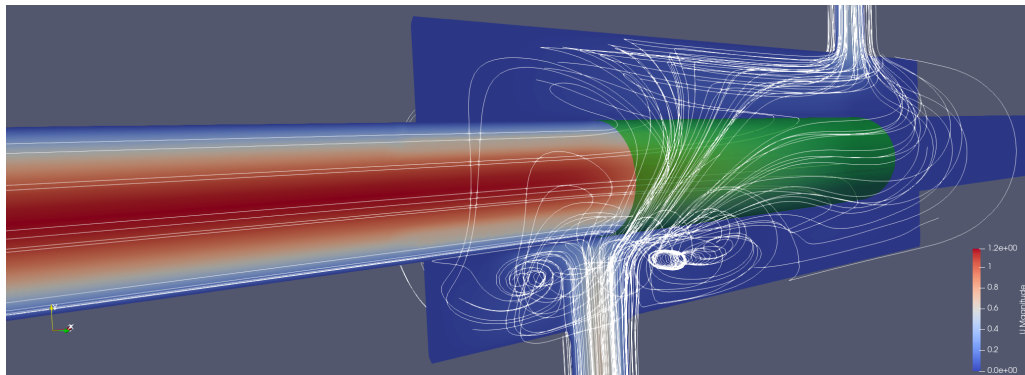
Experimental data to compare.

Table S1. Numerical values of the feed flow rates and the flow rates and compositions of CO₂ and CH₄ on the permeate and retentate sides with respect to the separation performance of CD-1-Cell under dry conditions at 30 °C shown in Figure 4a2,b2.

Total flow rate (mL·min ⁻¹)	Feed		Permeate side				Retentate side			
	Molar flow rate		Molar flow rate		Composition		Molar flow rate		Composition	
	CO ₂ (mol·s ⁻¹) × 10 ⁴	CH ₄ (mol·s ⁻¹) × 10 ⁴	CO ₂ (mol·s ⁻¹) × 10 ⁵	CH ₄ (mol·s ⁻¹) × 10 ⁷	CO ₂ (%)	CH ₄ (%)	CO ₂ (mol·s ⁻¹) × 10 ⁶	CH ₄ (mol·s ⁻¹) × 10 ⁵	CO ₂ (%)	CH ₄ (%)
1,000	3.41	3.41	9.27	1.95	99.79	0.21	248	34.1	42.14	57.86
800	2.73	2.73	8.87	2.05	99.77	0.23	184	27.3	40.32	59.68
700	2.39	2.39	8.73	2.10	99.76	0.24	151	23.9	38.84	61.16
600	2.05	2.05	8.41	2.11	99.75	0.25	121	20.4	37.10	62.90
500	1.71	1.71	8.05	2.18	99.73	0.27	90.1	17.0	34.59	65.41
400	1.36	1.36	7.50	2.33	99.69	0.31	61.5	13.6	31.12	68.88
300	1.02	1.02	6.58	2.51	99.62	0.38	36.6	10.2	26.42	73.58
200	0.68	0.68	5.10	2.82	99.45	0.55	17.4	6.79	20.41	79.59
100	0.34	0.34	2.95	3.32	98.89	1.11	4.88	3.38	12.63	87.37
50	0.17	0.17	1.62	3.65	97.80	2.20	1.17	1.67	6.54	93.46
25	0.09	0.09	0.87	3.81	95.80	4.20	0.17	0.82	2.02	97.98

Jeong et al., 2022.





Macroscopic Modeling - **Governing Equations**

Mass conservation equation:

$$\frac{\partial}{\partial t} (\rho) + \nabla \cdot (\rho \mathbf{U}) = 0$$

Momentum conservation equation:

$$\frac{\partial}{\partial t} (\rho \mathbf{U}) + \nabla \cdot (\rho \mathbf{U} \mathbf{U}) = -\nabla p + \nabla \cdot \left(\mu \left(\nabla \mathbf{U} + (\nabla \mathbf{U})^T \right) \right)$$

Energy conservation equation:

$$\frac{\partial}{\partial t} (\rho c_p T) + \nabla \cdot (\rho c_p T \mathbf{U}) = \nabla \cdot (k \nabla T)$$

Mass concentration equation:

$$\frac{\partial}{\partial t} (\rho Y_i) + \nabla \cdot (\rho Y_i \mathbf{U}) = \nabla \cdot (\rho D_{m,i} \nabla Y_i) + S_{m,i}$$

$$S_{m,i} = \frac{c_i A}{V_c} (p_{i,R} - p_{i,P})$$

Y_i : mass fraction of species i ; $D_{m,i}$: mass diffusion coefficient of species i ;
 $S_{m,i}$: mass transfer source term for species i .

The gas is modeled as an ideal gas:

$$pV = nR_uT$$

The viscosity is calculated using Sutherland's model:

$$\mu = \frac{A_s \sqrt{T}}{1 + T_s/T}$$

Thermal conductivity:

$$k = \mu c_v (1.32 + 1.77R/c_v)$$

The governing equations follow a standard pattern.

We can generalize this equation for any transported fluid property.

$$\frac{\partial (\rho\phi)}{\partial t} + \nabla \cdot (\rho \mathbf{U} \phi) = \nabla \cdot (\Gamma_\phi \nabla \phi) + S_\phi$$

$\frac{\partial(\rho\phi)}{\partial t}$ is the transient term, representing the rate of change of ϕ within the control volume.

$\nabla \cdot (\rho \mathbf{U} \phi)$ is the advective flux term, representing the transport of ϕ due to fluid flow.

$\nabla \cdot (\Gamma_\phi \nabla \phi)$ is the diffusive transport term, representing the transport of ϕ by diffusion.

S_ϕ is the source term, representing the rate of generation or destruction of ϕ , per unit volume.

Table of Contents

- 1 Membrane Gas Separation
- 2 CFD
- 3 OpenFOAM
- 4 Simulation: Lid-Driven Cavity
- 5 Simulation: 3D Lid-Driven Cavity
- 6 Homework
- 7 Conclusion
- 8 Supplementary Material
- 9 Papers

Computational Fluid Dynamics

What is CFD?

- Numerical approach to solve fluid flow problems
- Solves Navier-Stokes equations on a computational grid
- Provides detailed flow field information (velocity, pressure, temperature, species concentration)

Why use CFD?

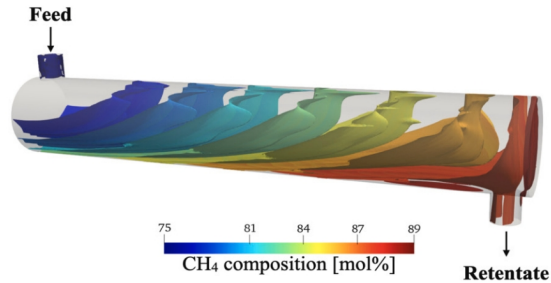
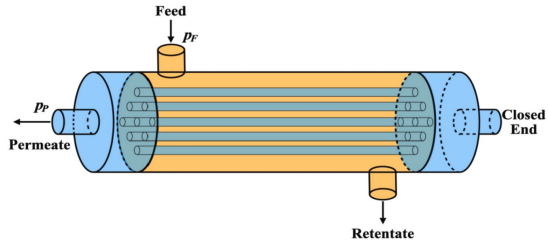
- Cheaper and faster than experiments for many cases
- Provides complete flow field data (not just point measurements)
- Enables simulation of conditions difficult to test experimentally

CFD-Driven Solutions in Membrane Gas Separation

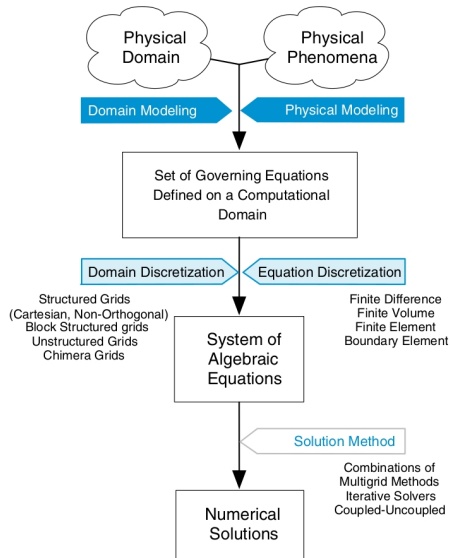
- Material Performance Analysis
 - Virtual testing of membrane materials under mixed-gas feeds
 - Plasticization prediction via polymer-fluid MD/CFD coupling
- Single Module Optimization
 - Dead zone identification through 3D velocity field mapping
 - Spacer design using parametric CFD studies
 - Transient modeling

CFD-Driven Solutions in Membrane Gas Separation

- Multi-Stage System Design
 - Cascade module interconnection analysis
 - Optimal stage cut determination per module
- Industrial Scale-Up
 - Full-plant digital twins with real boundary conditions
 - Tolerance analysis for manufacturing variations



Yao et al., 2025.



Moukalled et al., 2016.

Common Discretization Methods:

- **Finite Difference (FDM):**

- Approximates derivatives using Taylor series
- Simple, but less suitable for complex geometries

- **Finite Element (FEM):**

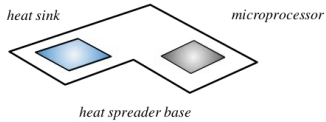
- Uses weighted residuals and shape functions
- Powerful for structural mechanics, but computationally expensive for fluid flows

- **Finite Volume (FVM):**

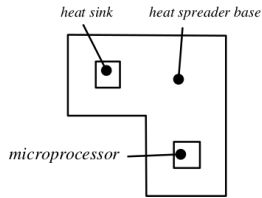
- Integrates equations over control volumes
- Conservative by construction (good for mass/momentum balance)
- Handles complex geometries well

Why Finite Volume Method (FVM) Dominates CFD?

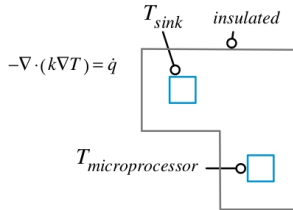
- **Conservation**
 - Naturally enforces mass, momentum, and energy balance
- **Flexibility**
 - Works well with unstructured meshes (complex geometries)
- **Robustness**
 - Stable for a wide range of flow regimes
- **Industry Standard**
 - Used in the most common CFD softwares



Domain Modeling



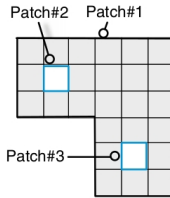
Physical Modeling



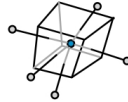
Moukalled et al., 2016.



Domain Discretization



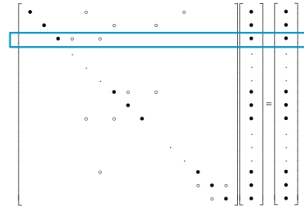
Equations Discretization



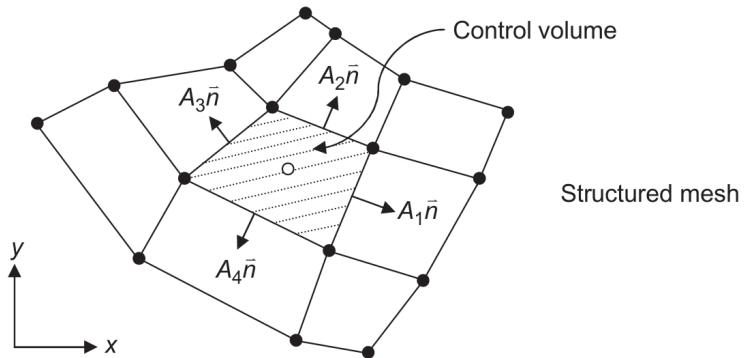
$$\underbrace{\frac{\partial(\rho\phi)}{\partial t}}_{\text{transient term}} + \underbrace{\nabla \cdot (\rho\mathbf{v}\phi)}_{\text{convection term}} = \underbrace{\nabla \cdot (\Gamma\nabla\phi)}_{\text{diffusion term}} + \underbrace{Q}_{\text{source term}}$$
$$a_C\phi_C + \sum_{F \sim NB(C)} a_F\phi_F = b_C$$

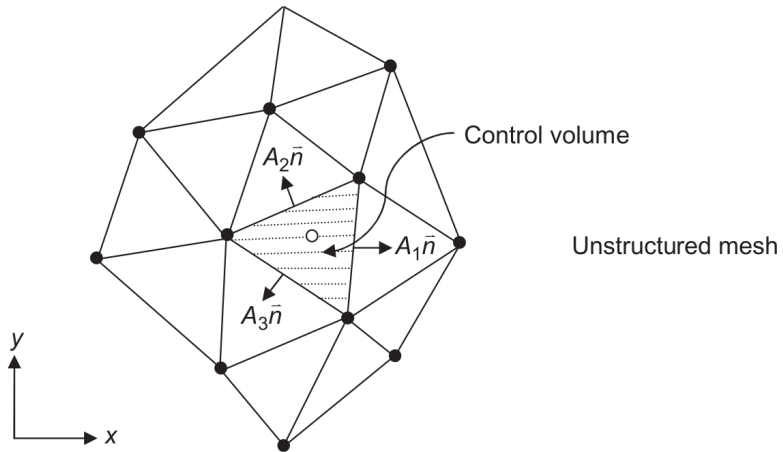


Solution Method



Moukalled et al., 2016.





FVM

The values of variables are stored at the centroid of each control volume.

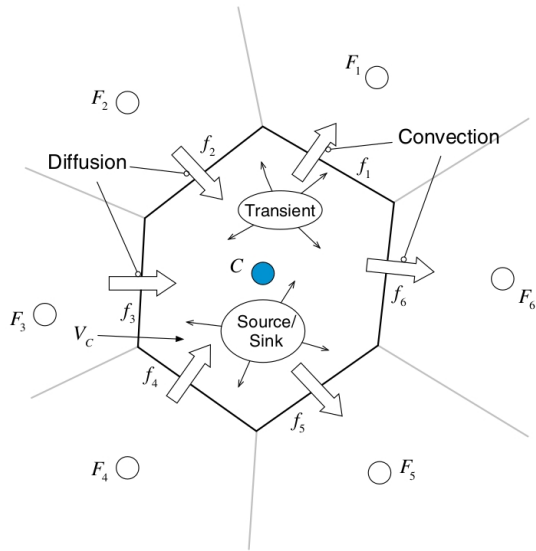
An interpolation scheme is used to express variable values at the control volume surfaces in terms of the centroid values. Suitable quadrature formulas are applied to approximate surface and volume integrals.

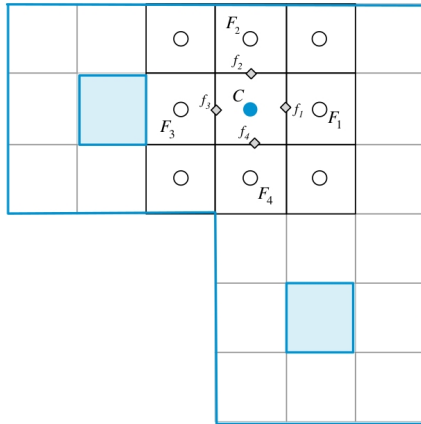
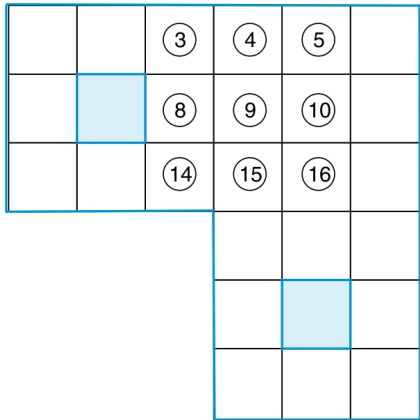
An algebraic equation is obtained for each control volume, incorporating values from neighboring cells.

The method is conservative, provided that surface integrals computed on a shared face are identical for the control volumes adjacent to that face.

A disadvantage of this method is that higher-than-second-order differentiation approximations are harder to develop in 3D.

This is due to the need for two levels of approximation: interpolation and integration.





Example: permanent energy equation for a solid with internal heat generation $\dot{q}_V$.

$$-\nabla \cdot (k \nabla T) = \dot{q}$$

$$-\int_{CV} \nabla \cdot (k \nabla T) dV = \int_{CV} \dot{q} dV$$

$$-\int_{CS} (k \nabla T) \cdot d\mathbf{S} = \dot{q}_C V$$

$$-\sum_f (k \nabla T)_f \cdot \mathbf{S}_f = \dot{q}_C V$$

CFD Workflow:

Geometry → Meshing → Setup → Solution → Post-processing

General CFD Simulation Workflow

- **0) Pre-processing**

- Geometry creation (CAD tools)
- Mesh generation (structured/unstructured)
- Mesh quality checks (skewness, aspect ratio, non-orthogonality)

- **1) Physics Setup**

- Define governing equations (Navier-Stokes, species transport, etc.)
- Set boundary/initial conditions
- Select numerical schemes (discretization, solvers)
- Set physical properties

• 2) Solution

- Run simulation (steady/transient)
- Monitor convergence (residuals, forces)
- Parallel computation (domain decomposition)

• 3) Post-processing

- Visualize results (velocity, pressure fields)
- Extract quantitative data (forces, fluxes)
- Validate against theory/experiments

CFD Softwares

OpenFOAM

Ansys Fluent

Ansys CFX

STAR-CCM+

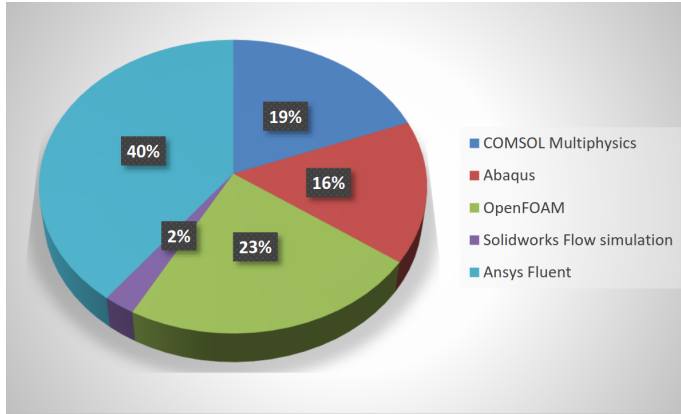
SU2

FLOW-3D

COMSOL Multiphysics

SIMSCALE





“Analysis of the CFD applications used in the scientific articles in the field of Energy Engineering Research, published in the Scopus database.”
Nely et al., 2021.

Why OpenFOAM?

- **Open-source**
Full access to source code
- **Free to use**
No licensing costs
- **Active community**
Forums, tutorials, and documentation
- **Widely adopted**
Used in academia and industry
- **Extensive features**
Advanced solvers for complex physics

Challenges in OpenFOAM:

- **Steep learning curve**

Requires some basic knowledge of Linux and maybe C++

- **No GUI**

Mostly command-line driven

- **Documentation gaps**

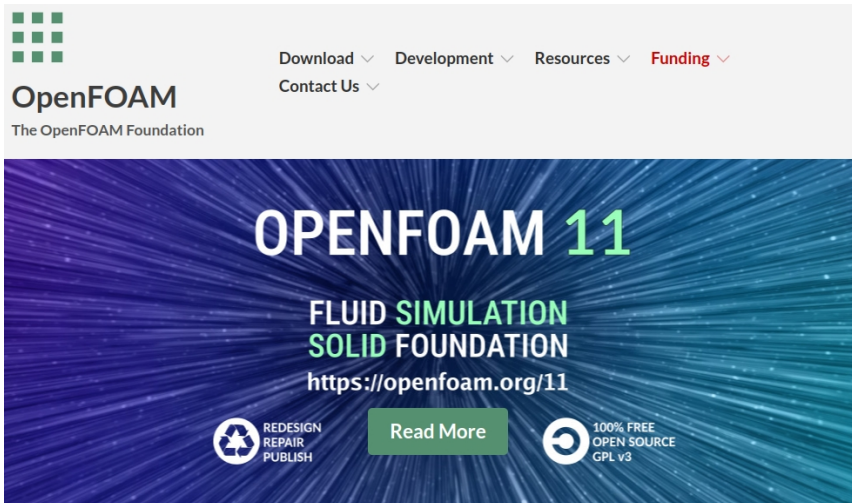
Some advanced and new features lack clear documentation

Table of Contents

- 1 Membrane Gas Separation
- 2 CFD
- 3 OpenFOAM**
- 4 Simulation: Lid-Driven Cavity
- 5 Simulation: 3D Lid-Driven Cavity
- 6 Homework
- 7 Conclusion
- 8 Supplementary Material
- 9 Papers

Open-source Field Operation And Manipulation

openfoam.org



The banner features a blue and green background with a radial light effect. At the top left is the OpenFOAM logo (a 3x3 grid of green squares). To its right are navigation links: 'Download', 'Development', 'Resources', 'Funding' (in red), and 'Contact Us', each followed by a downward arrow. Below the logo is the text 'OpenFOAM' and 'The OpenFOAM Foundation'. The main text 'OPENFOAM 11' is in large white and green letters. Below it, 'FLUID SIMULATION' and 'SOLID FOUNDATION' are in green. The URL 'https://openfoam.org/11' is in white. At the bottom left is the 'REDESIGN REPAIR PUBLISH' logo. In the center is a green 'Read More' button. At the bottom right is the '100% FREE OPEN SOURCE GPL v3' logo.

Download ▾ Development ▾ Resources ▾ Funding ▾
Contact Us ▾

OpenFOAM
The OpenFOAM Foundation

OPENFOAM 11

FLUID SIMULATION
SOLID FOUNDATION

<https://openfoam.org/11>

REDESIGN
REPAIR
PUBLISH

Read More

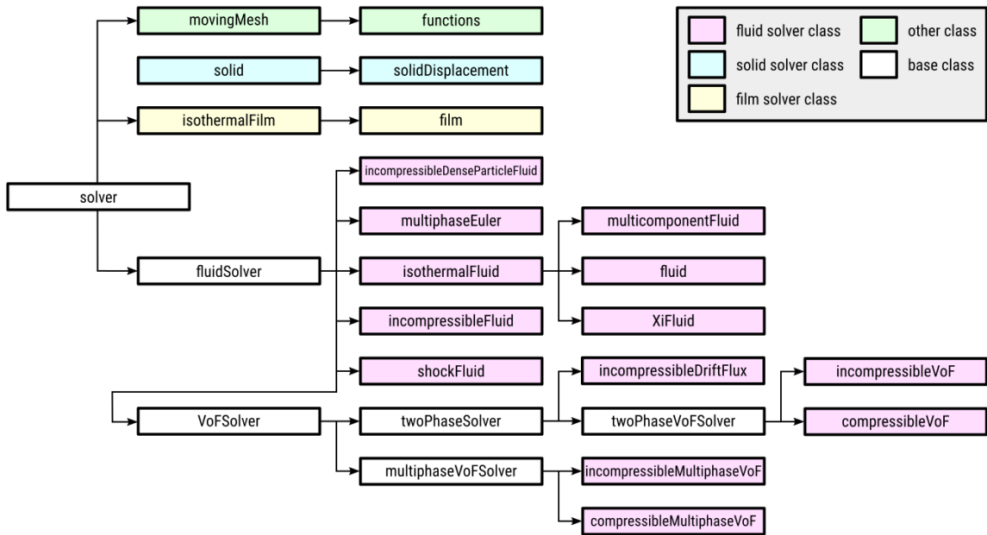
100% FREE
OPEN SOURCE
GPL v3



User Guide version 11

11th July 2023

<https://openfoam.org>



Code

Blame

59 lines (45 loc) · 1.86 KB

```

30 // * * * * * Member Functions * * * * *
31
32 void Foam::solvers::incompressibleFluid::momentumPredictor()
33 {
34     volVectorField& U(U_);
35
36     tUEqn =
37     (
38         fvm::ddt(U) + fvm::div(phi, U)
39         + MRF.DDt(U)
40         + momentumTransport->divDevSigma(U)
41         ==
42         fvModels().source(U)
43     );
44     fvVectorMatrix& UEqn = tUEqn.ref();
45
46     UEqn.relax();
47
48     fvConstraints().constrain(UEqn);
49
50     if (pimple.momentumPredictor())
51     {
52         solve(UEqn == -fvc::grad(p));
53
54         fvConstraints().constrain(U);
55     }
56 }

```


Code

Blame

109 lines (8...

```
42     );  
43  
44     reaction->correct();  
45  
46     forAll(Y, i)  
47     {  
48         if (composition.solve(i))  
49         {  
50             volScalarField& Yi = Y_[i];  
51  
52             fvScalarMatrix YiEqn  
53             (  
54                 fvm::ddt(rho, Yi)  
55                 + mvConvection->fvmDiv(phi, Yi)  
56                 + thermophysicalTransport->divj(Yi)  
57                 ==  
58                 reaction->R(Yi)  
59                 + fvModels().source(rho, Yi)  
60             );  
61  
62             YiEqn.relax();  
63  
64             fvConstraints().constrain(YiEqn);  
65  
66             YiEqn.solve("Yi");  
67  
68             fvConstraints().constrain(Yi);  
69         }  
70     }  
71  
72     composition.normalise();  
73
```

Code

Blame

218 lines (1...

```

102     }
103
104
105     // * * * * * Member Functions * * * * * //
106
107     template<class Thermo>
108     inline Foam::scalar Foam::sutherlandTransport<Thermo>::mu
109     (
110         const scalar p,
111         const scalar T
112     ) const
113     {
114         return As_::sqrt(T)/(1.0 + Ts_/T);
115     }
116
117
118     template<class Thermo>
119     inline Foam::scalar Foam::sutherlandTransport<Thermo>::kappa
120     (
121         const scalar p,
122         const scalar T
123     ) const
124     {
125         scalar Cv_ = this->Cv(p, T);
126         return mu(p, T)*Cv_*(1.32 + 1.77*this->R()/Cv_);
127     }
128
129
130     // * * * * * Member Operators * * * * * //
131

```

Code

Blame

154 lines (125 loc) · 4.15 KB

```
70     forAll(owner, facei)
71     {
72         GradType Sfssf = Sf[facei]*issf[facei];
73
74         igGrad[owner[facei]] += Sfssf;
75         igGrad[neighbour[facei]] -= Sfssf;
76     }
77
78     forAll(mesh.boundary(), patchi)
79     {
80         const fvPatch& p = mesh.boundary()[patchi];
81         const labelList& pFaceCells = p.faceCells();
82         const vectorField& pSf = mesh.Sf().boundaryField()[patchi];
83         const fvsPatchField<Type>& pssf = ssf.boundaryField()[patchi];
84
85         forAll(p, facei)
86         {
87             igGrad[pFaceCells[facei]] += pSf[facei]*pssf[facei];
88         }
89     }
90
91     igGrad /= mesh.V();
92
93     gGrad.correctBoundaryConditions();
94
95     return tgGrad;
96 }
```

Code Blame 479 lines (4...

```
167
168
169     template<class EquationOfState>
170     inline Foam::scalar Foam::janafThermo<EquationOfState>::Cp
171     (
172         const scalar p,
173         const scalar T
174     ) const
175     {
176         const coeffArray& a = coeffs(T);
177         return
178             (((a[4]*T + a[3])*T + a[2])*T + a[1])*T + a[0]
179             + EquationOfState::Cp(p, T);
180     }
181
```

We will use OpenFOAM 11 in this course.

OpenFOAM is developed for Ubuntu. To download and install OpenFOAM 11 in Ubuntu, you can check [this page](#).

To install and use OpenFOAM 11 in Windows, you can check this [tutorial](#).

General OpenFOAM Workflow

This is what we are going to do.

- **0) Pre-processing**

- Geometry: `blockMesh` (simple geometries) or `gmsh`
- Meshing: `blockMesh` (simple geometries), `gmsh` or `snappyHexMesh` (complex geometries)
- Mesh quality: `checkMesh` and visualization

- **1) Case Setup**

- Directory structure: `0/`, `constant/`, `system/`
- Boundary and initial conditions: Define in `0/` files (e.g., `p`, `U`, `T`)
- Physical properties: `constant/`
- Solver settings: `constant/ controlDict`, `fvSchemes`, `fvSolution`

• 2) Running the Simulation

- Solver execution: `foamRun`
- Parallel runs: `decomposePar + mpirun -np 4 foamRun -parallel`
- Monitoring:
`foamMonitor -l postProcessing/residuals/0/residuals.dat`

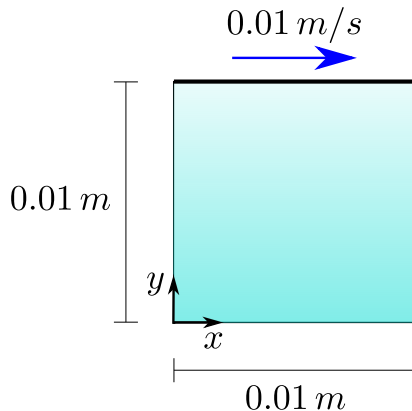
• 3) Post-processing

- Visualization: `paraview`
- Data extraction: `postProcess` utility and `ParaView`
- Automation: Python scripts

Table of Contents

- 1 Membrane Gas Separation
- 2 CFD
- 3 OpenFOAM
- 4 Simulation: Lid-Driven Cavity**
- 5 Simulation: 3D Lid-Driven Cavity
- 6 Homework
- 7 Conclusion
- 8 Supplementary Material
- 9 Papers

Let us begin our CFD studies with the classical **lid-driven cavity** problem.



In this case, we will simulate the behavior of a Newtonian, incompressible fluid in a square cavity with side length 0.01 m .

The left, right, and bottom walls are stationary. The top wall moves to the right with a velocity of 0.01 m/s .

We will consider the fluid's kinematic viscosity ν to be $10^{-6}\text{ m}^2/\text{s}$, which corresponds to the water viscosity.

$$\nu = \frac{\mu}{\rho}$$

Solving this problem means determining the pressure p and velocity $\mathbf{U}$ fields at every point in the domain and at each time instant.

We will solve the mass conservation equation,

$$\nabla \cdot \mathbf{U} = 0 ,$$

and the Navier-Stokes equations,

$$\frac{\partial \mathbf{U}}{\partial t} + \nabla \cdot (\mathbf{U}\mathbf{U}) = -\nabla \left(\frac{P}{\rho} \right) + \nabla \cdot (\nu \nabla \mathbf{U}) ,$$

considering an incompressible flow.

Let's start.

First, copy the folder

/opt/openfoam11/tutorials/incompressibleFluid/cavity

to a directory of your choice.

In OpenFOAM, each simulation is contained in its own directory.

Rename this folder to *lidDrivenCavity*.

There are 3 folders (*directories*) within the *lidDrivenCavity* directory:

- 0
- constant
- system

This is the basic structure for all OpenFOAM simulations.

In the **0 (ZERO)** directory, we store the initial and boundary conditions for all primitive variables being solved.

The **constant** directory contains the physical properties of the simulation (viscosity, density, etc.)

The **system** directory holds parameters controlling the simulation (start time, time step, end time, ...), numerical schemes, convergence criteria, and other numerical settings.

In CFD, every simulation is divided into 3 stages: **pre-processing**, **processing**, and **post-processing**.

Pre-processing: Problem setup (geometry, mesh, initial conditions and boundary conditions).

Processing: Numerical solution of the governing equations.

Post-processing: Results analysis and visualization.

Let's begin with **mesh generation**.

For this lid-driven cavity example (a simple square), we'll use *blockMesh* (included with OpenFOAM) to generate the mesh.

We'll examine the *system/blockMeshDict* file.

Dict stands for dictionary - OpenFOAM's structured data format for configuration.

system/blockMeshDict

```
1 convertToMeters 0.01;
2
3 vertices
4 (
5     (0 0 0)      // 0
6     (1 0 0)      // 1
7     (1 1 0)      // 2
8     (0 1 0)      // 3
9     (0 0 0.1)    // 4
10    (1 0 0.1)    // 5
11    (1 1 0.1)    // 6
12    (0 1 0.1)    // 7
13 );
14
15 blocks
16 (
17     hex (0 1 2 3 4 5 6 7) (20 20 1) simpleGrading (1 1 1)
18 );
```

```

19 boundary
20 (
21     movingWall
22     {
23         type wall;
24         faces
25         (
26             (3 7 6 2)
27         );
28     }
29     fixedWalls
30     {
31         type wall;
32         faces
33         (
34             (0 4 7 3)
35             (2 6 5 1)
36             (1 5 4 0)
37         );
38     }

```

```

39     frontAndBack
40     {
41         type empty;
42         faces
43         (
44             (0 3 2 1)
45             (4 5 6 7)
46         );
47     }
48 );

```

Note that I've made some modifications to our simulation compared to the original tutorial case.

Ensure your case matches the setup shown here.

In the *blockMeshDict* file, I only changed the *convertToMeters* parameter to 0.01.

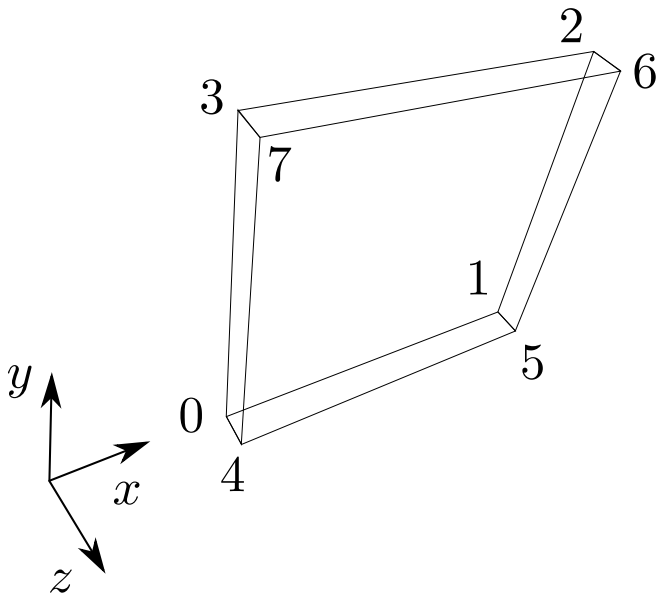
The *convertToMeters* parameter acts as a scaling factor - it converts the dimensions specified in the file to actual generated values. Our square cavity is 0.01 *m* in size.

After *convertToMeters*, we define the list of vertices for our geometry. The number preceding each vertex is just a comment for identification purposes. Anything following double slashes *//* in a line is ignored by OpenFOAM. These comments help with code readability.

We want to simulate a square (2D geometry). However, all OpenFOAM simulations are inherently three-dimensional.

Therefore, we'll create a parallelepiped (3D) but specify that we don't want to solve the problem in one direction (empty).

The z-direction thickness is unimportant since this is a 2D problem (in x and y). We're using 0.001 m for the z-dimension.



Using the vertex list, we'll now create a mesh block (or *block*).

This block consists of hexahedral cells, with:

- 20 cells in the x -direction
- 20 cells in the y -direction
- Only 1 cell in the z -direction

The *simpleGrading* parameter controls the cell size ratio. We'll keep it at its default value for now.

(Later, try changing the *simpleGrading* values to 1.2 and 0.8 to observe the effects.)

Next comes an important step: the definition of **boundary condition** types we'll apply to each surface.

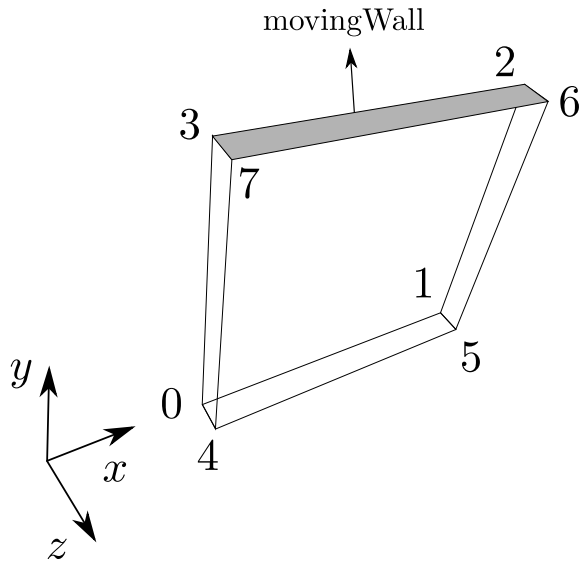
Every surface must belong to one of the defined boundary groups.

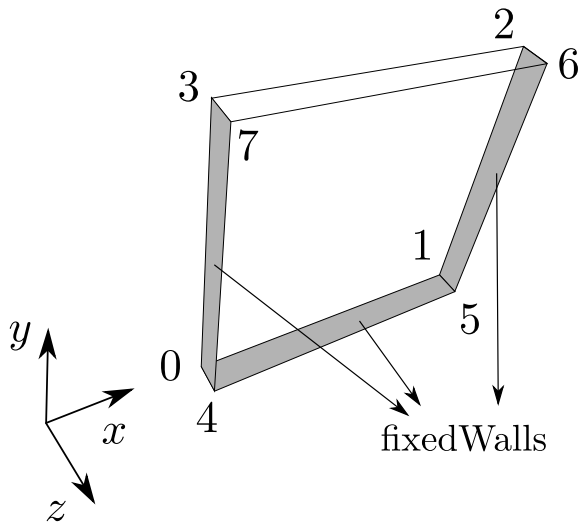
In this case, we have 3 boundary surface groups: *movingWall*, *fixedWalls*, and *frontAndBack*.

movingWall and *fixedWalls* are of type *wall*, while *frontAndBack* are of type *empty*.

The *empty* type indicates we don't want to solve the problem in the direction perpendicular to these surfaces.

Important: Be very careful when defining your problem's boundary conditions. This is a critical aspect of the simulation that significantly affects the final results.





Everything ready! Let's generate the mesh.

The terminal command to generate the mesh using *blockMeshDict* is:

```
$ blockMesh
```

After executing the command, the ***constant/polyMesh*** directory is created containing all mesh information.

Carefully examine the created files. Next we'll review the *boundary* file.

constant/polyMesh/boundary

```
1 3
2 (
3   movingWall
4   {
5       type            wall;
6       inGroups        List<word> 1(wall);
7       nFaces          20;
8       startFace       760;
9   }
10  fixedWalls
11  {
12      type            wall;
13      inGroups        List<word> 1(wall);
14      nFaces          60;
15      startFace       780;
16  }
```

```
17     frontAndBack
18     {
19         type          empty;
20         inGroups      List<word> 1(empty);
21         nFaces        800;
22         startFace     840;
23     }
24 )
```

To verify the mesh quality, execute the command:

```
$ checkMesh
```

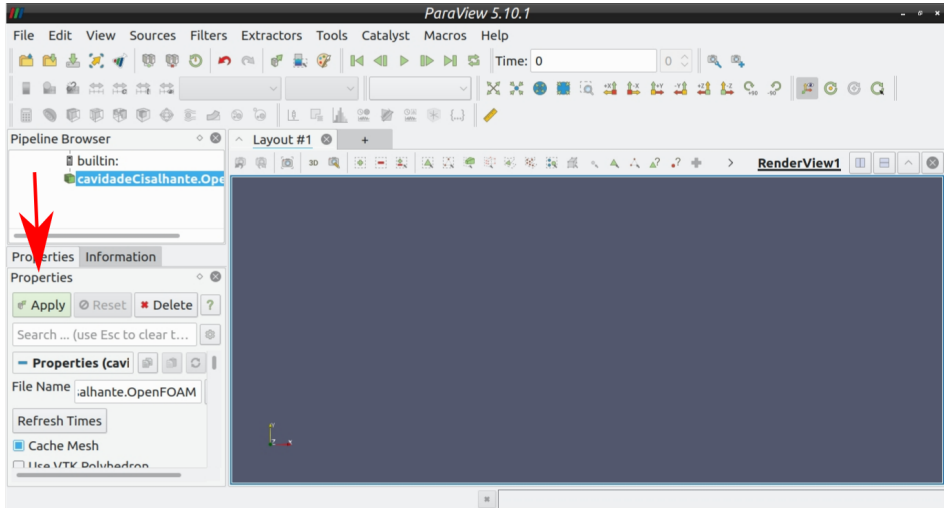
and analyze the output information.

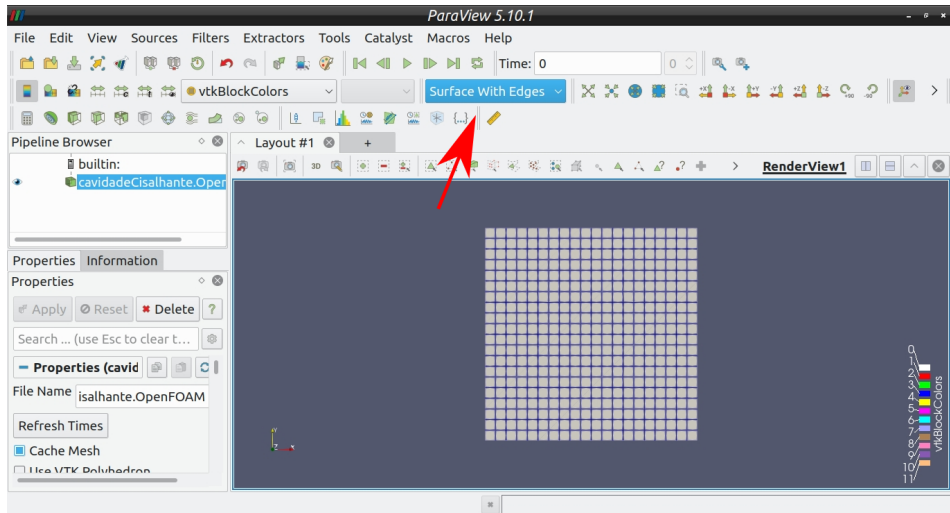
Key Parameters to Analyze: **Non-orthogonality** (should be $< 70^\circ$ for most cases); **Skewness** (ideal: < 5); **Aspect ratio** (should be < 1000).

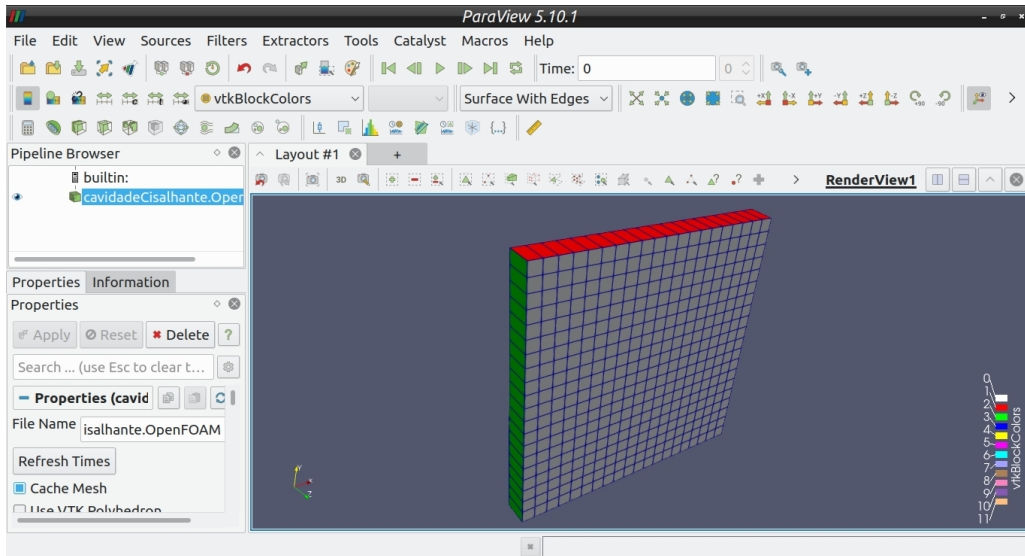
You can also visualize the generated mesh by running:

```
$ paraFoam
```

ParaView will launch. Click *Apply*, then change the display mode from *Surface* to *Surface With Edges*.







Mesh complete! You can close ParaView for now.

Let's proceed to the physical properties.

The *constant* directory contains two key files:

- *momentumTransport*
- *physicalProperties*

In *momentumTransport*, we define:

- Turbulence model (none for our laminar simulation)
- Viscosity model (Newtonian - the default setting)

Note: Flow Regimes in CFD

- **Laminar flow:** Ordered, predictable motion ($Re < 2300$ for pipes)
- **Turbulent flow:** Chaotic, 3D fluctuations ($Re > 4000$ for pipes)

Why we focus on laminar flow:

- Membrane separation typically operates in laminar regime
- Turbulence modeling requires:
 - Additional transport equations ($k-\epsilon$, $k-\omega$, etc.)
 - Wall functions
 - Higher computational cost
- Beyond this course's scope (covered in a complete CFD course)

constant/momentumTransport

```
1 FoamFile
2 {
3     format      ascii;
4     class       dictionary;
5     location     "constant";
6     object       momentumTransport;
7 }
8 // * * * * *
9
10 simulationType laminar;
```

In the *physicalProperties* file, we will define the value of kinematic viscosity ν (nu).

Kinematic viscosity is defined as the ratio of dynamic viscosity μ to fluid density ρ :

$$\nu = \frac{\mu}{\rho} .$$

The code in the *physicalProperties* file is shown on the next slide.

constant/physicalProperties

```
1 FoamFile
2 {
3     format      ascii;
4     class       dictionary;
5     location     "constant";
6     object       physicalProperties;
7 }
8 // * * * * *
9
10 viscosityModel constant;
11
12 nu              [0 2 -1 0 0 0 0] 1e-06;
13
14 // ***** //
```

First, we define the *viscosityModel* as *constant*. This simulation is for a Newtonian fluid, which has constant viscosity.

Next, we specify the value of kinematic viscosity *nu*.

In this case, *nu* is set to $10^{-6} \text{ m}^2/\text{s}$.

All variables in OpenFOAM are dimensional.

When introducing any variable, we must define both its dimensions and value.

The viscosity units are specified in square brackets.

In the SI system, the base units are represented as:

$$[kg \ m \ s \ K \ mol \ A \ cd]$$

These represent the 7 fundamental dimensions: mass (kg), length (m), time (s), temperature (K), amount of substance (mol), electric current (A), and luminous intensity (cd).

The kinematic viscosity dimensions are:

$$[0 \ 2 \ -1 \ 0 \ 0 \ 0 \ 0]$$

which corresponds to m^2/s .

The kinematic viscosity value is $10^{-6} \text{ m}^2/\text{s}$. The governing parameter of this problem (and the most important one in Fluid Mechanics) is the Reynolds number Re , defined as:

$$Re = \frac{UL}{\nu} ,$$

where U is the top wall velocity and L is the cavity size.

With $U = 0.01 \text{ m/s}$ (to be defined shortly) and $L = 0.01 \text{ m}$, our problem's Reynolds number is:

$$Re = \frac{0.01 \times 0.01}{10^{-6}} = 100$$

With the fluid properties and turbulence model defined (laminar in this case), we now proceed to the initial and boundary conditions.

Directory 0 (ZERO).

This directory contains the initial and boundary conditions for all primitive variables that are being solved.

Let's begin with the velocity field U .

```
1 FoamFile
2 {
3     format      ascii;
4     class       volVectorField;
5     object      U;
6 }
7
8 dimensions      [0 1 -1 0 0 0 0];
9
10 internalField uniform (0 0 0);
11
12 boundaryField
13 {
14     movingWall
15     {
16         type      fixedValue;
17         value      uniform (0.01 0
18                             0);
19     }
```

```
19     fixedWalls
20     {
21         type      noSlip;
22     }
23
24     frontAndBack
25     {
26         type      empty;
27     }
28 }
```

The velocity field is a *volVectorField*, meaning it's a vector field with values defined at cell centers.

The dimensions of velocity are m/s , which corresponds to:

$$[0 \ 1 \ -1 \ 0 \ 0 \ 0 \ 0]$$

The *internalField* variable specifies the initial value throughout the domain. Here we'll set it uniformly to zero.

Remember that velocity is a vector, so we use $(0 \ 0 \ 0)$ notation.

Next we have a dictionary called *boundaryField*, where we must specify conditions for all domain boundaries.

The boundary condition for U on the *movingWall* group is of type *fixedValue*, set to $(0.01 \ 0 \ 0)$.

This defines the velocity at the top wall: 0.01 m/s in the x -direction and zero in other directions.

For surfaces designated as *fixedWalls*, we specify zero velocity at their contact - this is the no-slip condition (*noSlip*).

This condition is commonly used for wall boundaries.

Finally, since our problem is two-dimensional, we set the *frontAndBack* surfaces to *empty* boundary condition. This means we won't solve the problem in the direction perpendicular to these walls (z-direction).

Now we'll configure the **pressure** field.

0/p

```
1 FoamFile
2 {
3     format      ascii;
4     class       volScalarField;
5     object      p;
6 }
7
8 dimensions      [0 2 -2 0 0 0 0];
9
10 internalField uniform 0;
11
12 boundaryField
13 {
14     movingWall
15     {
16         type      zeroGradient;
17     }
```

```
18     fixedWalls
19     {
20         type      zeroGradient;
21     }
22     frontAndBack
23     {
24         type      empty;
25     }
26 }
```

Pressure is a *volScalarField*: a scalar field.

All incompressible OpenFOAM solvers use modified pressure, defined as pressure divided by density:

$$p = \frac{P}{\rho}$$

The dimensions of modified pressure are m^2/s^2 , corresponding to:

$$[0 \ 2 \ -2 \ 0 \ 0 \ 0 \ 0]$$

The initial pressure in the cavity is set to 0 (relative pressure). For incompressible flow simulations, only pressure variation matters, not absolute values.

The most common pressure boundary condition at solid walls is *zeroGradient* (zero normal gradient).

This condition applies to both the moving top wall (*movingWall*) and the stationary walls (*fixedWalls*).

For the *frontAndBack* surfaces, we again implement the *empty* condition.

Note:

In the 0 directory, you may delete the following turbulence-related files:

- `epsilon` (turbulent dissipation rate)
- `k` (turbulent kinetic energy)
- `nut` (turbulent viscosity)
- `nuTilda` (modified turbulent viscosity)
- `omega` (specific dissipation rate)

These files are only required for turbulent simulations. For laminar flow (this case), they are unnecessary.

We now have:

- The computational mesh
- Physical properties
- Initial and boundary conditions

Next, we configure the numerical parameters in the ***system*** directory.

The *controlDict* file contains the simulation's control settings.

system/controlDict

```
1 FoamFile
2 {
3     format      ascii;
4     class       dictionary;
5     location     "system";
6     object       controlDict;
7 }
8 // * * * * *
9
10 application    foamRun;
11
12 solver         incompressibleFluid;
13
14 startFrom      startTime;
15
16 startTime      0;
17
18 stopAt         endTime;
19
20 endTime        2.0;
```

```
21 deltaT      0.001;
22
23 writeControl adjustableRunTime;
24
25 writeInterval 0.2;
26
27 purgeWrite   0;
28
29 writeFormat   ascii;
30
31 writePrecision 6;
32
33 writeCompression off;
34
35 timeFormat     general;
36
37 timePrecision  6;
38
39 runTimeModifiable true;
```

For this simulation, we're using the *foamRun* application with the *incompressibleFluid* solver.

The simulation starts at *startTime* = 0.

It runs until *endTime* = 2.0.

The time step (Δt) is set to 0.001 s in the *deltaT* parameter.

This configuration will execute 2000 time steps in total.

After ΔT , we configure the output file parameters.

The *writeControl* is set to *adjustableRunTime*. This adjusts the time step to ensure output is written at each interval specified by *writeInterval*.

In our case, we want results written every 0.2 s . With the simulation running from 0 to 2 s , this will generate 10 solution directories ($0, 0.2, 0.4, \dots$).

We will maintain the default settings for: *purgeWrite*, *writeFormat*, *writePrecision*, *writeCompression*, *timeFormat*, *timePrecision*, and *runTimeModifiable*.

Some of these settings will in the *controlDict* file will be covered in more depth in future lectures.

In the ***system*** directory, we also have two key files:

- *fvSchemes* - Defines the finite volume discretization schemes (temporal and spatial integration methods)
- *fvSolution* - Specifies the solution algorithms, linear solvers, and convergence tolerances

Note these files have been modified from the original tutorial case.

We'll examine these files in detail in upcoming lessons, but for now we'll proceed with the current configurations.

system/fvSchemes

```
1 ddtSchemes
2 {
3     default      Euler;
4 }
5
6 gradSchemes
7 {
8     default      Gauss linear;
9     grad(p)      Gauss linear;
10 }
11
12 divSchemes
13 {
14     default      none;
15     div(phi,U)   Gauss linear;
16 }
```

```
17 laplacianSchemes
18 {
19     default      Gauss linear orthogonal;
20 }
21
22 interpolationSchemes
23 {
24     default      linear;
25 }
26
27 snGradSchemes
28 {
29     default      orthogonal;
30 }
```

system/fvSolution

```
1 solvers
2 {
3     p
4     {
5         solver          PCG;
6         preconditioner  DIC;
7         tolerance       1e-06;
8         relTol          0.05;
9     }
10
11    pFinal
12    {
13        $p;
14        relTol          0;
15    }
```

```
16
17    "(U|UFinal)"
18    {
19        solver          smoothSolver;
20        smoother        symGaussSeidel;
21        tolerance       1e-05;
22        relTol          0;
23    }
24 }
25
26 PIMPLE
27 {
28     nCorrectors        2;
29     nNonOrthogonalCorrectors 0;
30     pRefCell            0;
31     pRefValue           0;
32 }
```

All set! We're now ready to run the simulation.

To execute the simulation: Run the command

```
$ foamRun
```

The terminal will display real-time simulation progress and diagnostic messages, helping you monitor the calculation status and identify potential issues.

```
// * * * * *  
Create time  
  
Create mesh for time = 0  
  
Selecting solver incompressibleFluid  
Selecting viscosity model constant  
Selecting turbulence model type laminar  
Selecting laminar stress model Stokes  
No MRF models present  
  
Courant Number mean: 0 max: 0  
  
PIMPLE: No convergence criteria found  
  
PIMPLE: Operating solver in transient mode with 1 outer corrector  
PIMPLE: Operating solver in PISO mode  
  
Starting time loop
```

```
Courant Number mean: 0 max: 0  
Time = 0.001s
```

```
smoothSolver: Solving for Ux, Initial residual = 1, Final residual = 7.86019e-06, No Iterations 13  
smoothSolver: Solving for Uy, Initial residual = 0.990973, Final residual = 7.08708e-06, No Iterations 11  
GAMG: Solving for p, Initial residual = 1, Final residual = 0.0300802, No Iterations 3  
time step continuity errors : sum local = 1.64727e-05, global = 1.74701e-20  
GAMG: Solving for p, Initial residual = 0.326659, Final residual = 4.67831e-07, No Iterations 13  
time step continuity errors : sum local = 4.9902e-10, global = 1.4426e-20, cumulative = 1.4426e-20  
ExecutionTime = 0.049312 s ClockTime = 0 s
```

```
Courant Number mean: 0.0427361 max: 0.169311  
Time = 0.05s
```

```
smoothSolver: Solving for Ux, Initial residual = 0.000790653, Final residual = 5.11792e-06, No Iterations 6  
smoothSolver: Solving for Uy, Initial residual = 0.00119873, Final residual = 7.88519e-06, No Iterations 6  
GAMG: Solving for p, Initial residual = 0.000489861, Final residual = 3.24376e-05, No Iterations 2  
time step continuity errors : sum local = 3.91778e-08, global = -4.9879e-20  
GAMG: Solving for p, Initial residual = 0.00021758, Final residual = 9.09843e-07, No Iterations 4  
time step continuity errors : sum local = 1.27091e-09, global = 1.82459e-19, cumulative = -3.02665e-20  
ExecutionTime = 0.106746 s ClockTime = 0 s
```

```
End
```

A critical value displayed during the simulation is the **Courant number**, defined as:

$$C = \frac{\Delta t U}{\Delta x}$$

The Courant number represents the ratio of physical speed to numerical speed.

For simulation stability, maintain the *max Courant number* below or near 1.

Note: Try running a simulation with $C > 1$ to observe stability issues.

The simulation has completed. Multiple directories were generated, each representing results at a specific time step (named after the time value).

Each directory contains:

- Velocity field (U)
- Pressure field (p)
- Surface volumetric flux (ϕ)

Additionally, a *uniform* directory contains supplementary time-step information.

This output structure enables restarting simulations from any saved time state.

With the simulation results ready, we begin **post-processing**.

For this example, we'll visualize results using ParaView. The following slides demonstrate key visualization steps.

If ParaView isn't open, run this command in your case directory:

```
$ paraFoam
```

Note: Windows ParaView Setup

Install ParaView from paraview.org/download.

Visualization Steps:

- 1 First ensure the `case.foam` file exists:
 - If missing, create it manually (empty file)
 - Right-click in case folder → New → Text Document
 - Rename to `case.foam` (remove `.txt` extension)
- 2 Open ParaView via Start Menu
- 3 Go to *File* → *Open...*
- 4 Set file type to *OpenFOAM*
- 5 Select `case.foam`
- 6 Click *Apply*

The `case.foam` file acts as a reader for OpenFOAM's case structure.

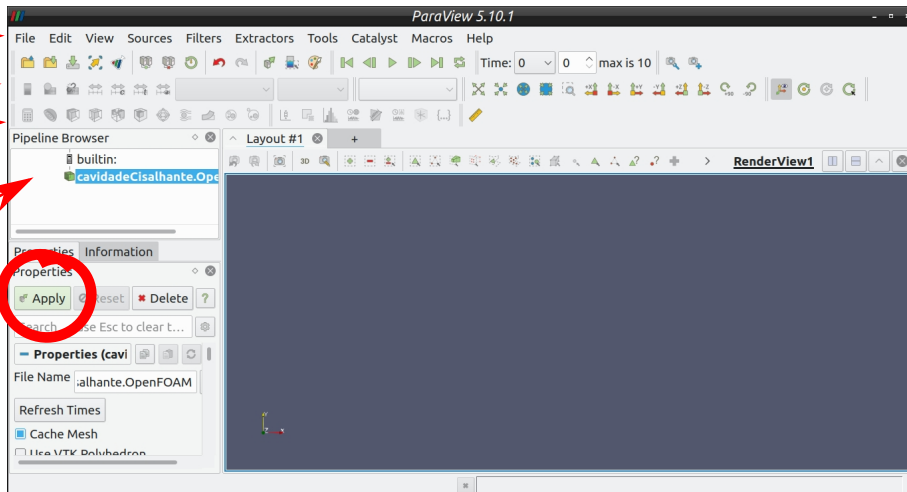
Click *Apply*.

Menu Bar →

Toolbars →

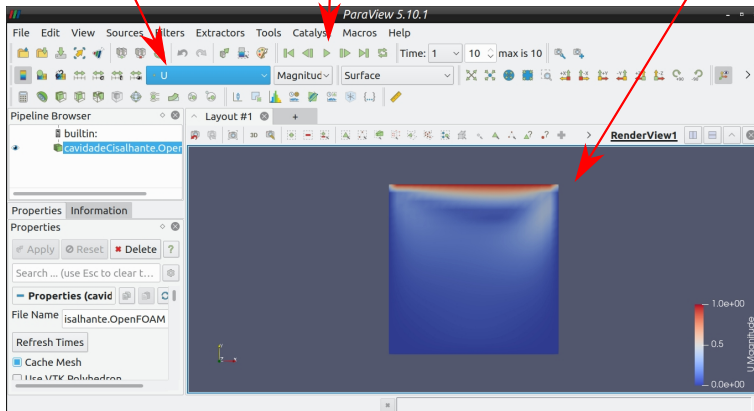
Filters →

Pipeline Tree →



To visualize the velocity and pressure contours.

1. Choose U
2. Click Play
3. Use the mouse to change the camera



4. Change from U to p and click play again

Now we will visualize the flow streamlines.

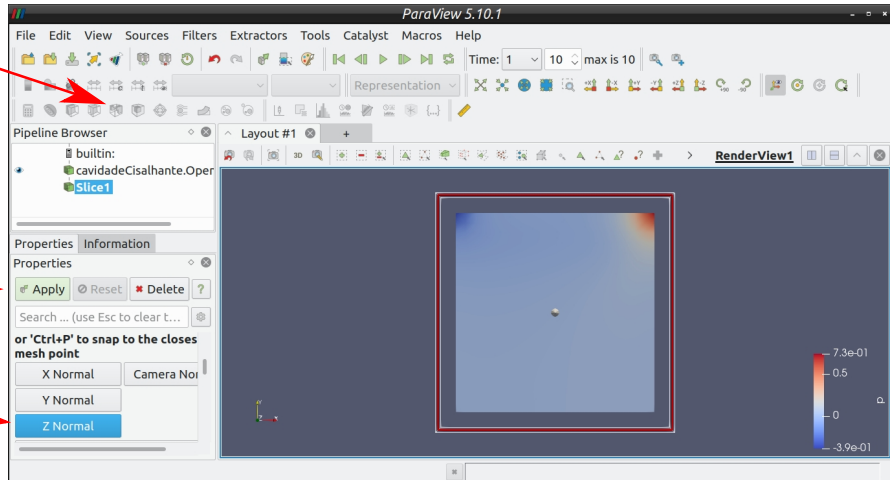
1. Slice



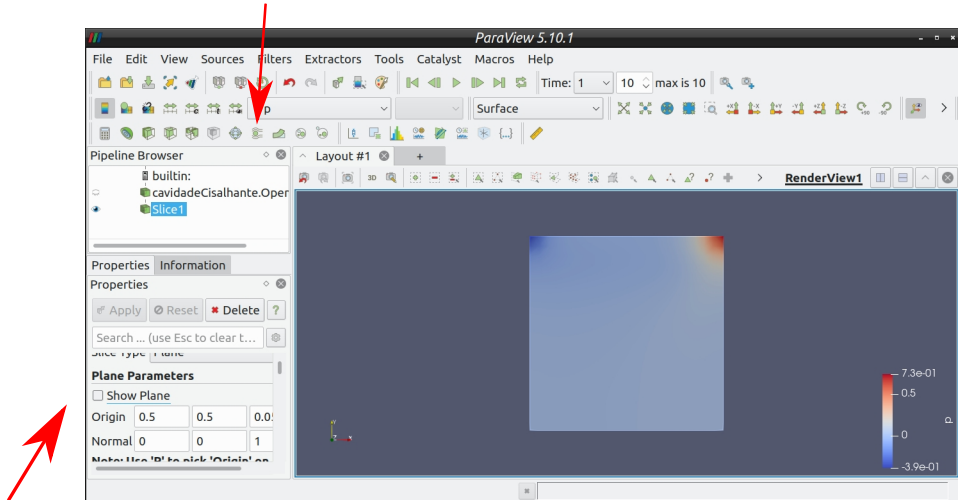
3. Apply



2. Z Normal



5. Click in Stream Tracer



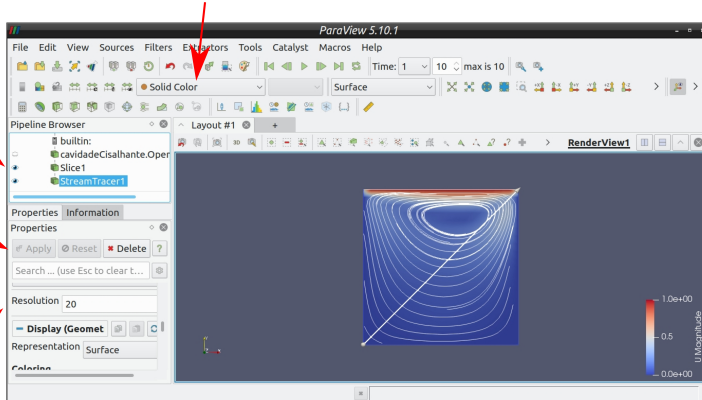
4. Uncheck the option Show Plane

8. Choose Solid Color for StreamTracer1

9. Click in the eye of Slice1

7. Click in Apply

6. Put 20 in Resolution



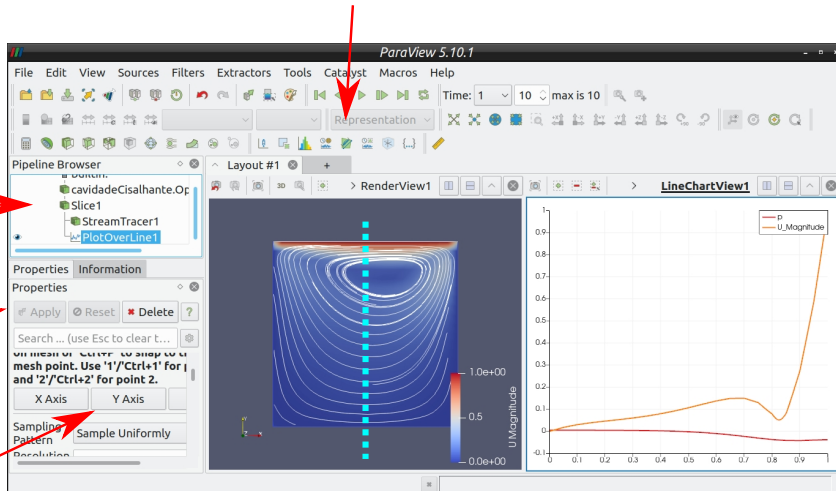
Now we are going to plot U and p as a function of y in the center of the cavity ($x = 0.005\text{ m}$).

2. Plot Over Line

1. Select Slice

4. Apply

3. Y Axis



To export results, navigate to *File* → *Save Screenshot...*

This demonstrates only a basic ParaView workflow. ParaView offers extensive advanced capabilities.

You can further refine visualizations to achieve publication-quality figures.

Table of Contents

- 1 Membrane Gas Separation
- 2 CFD
- 3 OpenFOAM
- 4 Simulation: Lid-Driven Cavity
- 5 Simulation: 3D Lid-Driven Cavity**
- 6 Homework
- 7 Conclusion
- 8 Supplementary Material
- 9 Papers

To transform our 2D cavity into a 3D simulation, modify the *frontAndBack* boundary treatments:

Step 1: Mesh Modification (*blockMeshDict*)

- Extend vertex coordinates to 0.01 m in z -direction
- Set 20 cells in z -direction
- Change boundary type from *empty* to *wall*

Step 2: Velocity Field ($0/U$)

- Update *frontAndBack* type to *noSlip*

Step 3: Pressure Field ($0/p$)

- Change *frontAndBack* to *zeroGradient*

Execute the case and analyze the 3D results in ParaView.

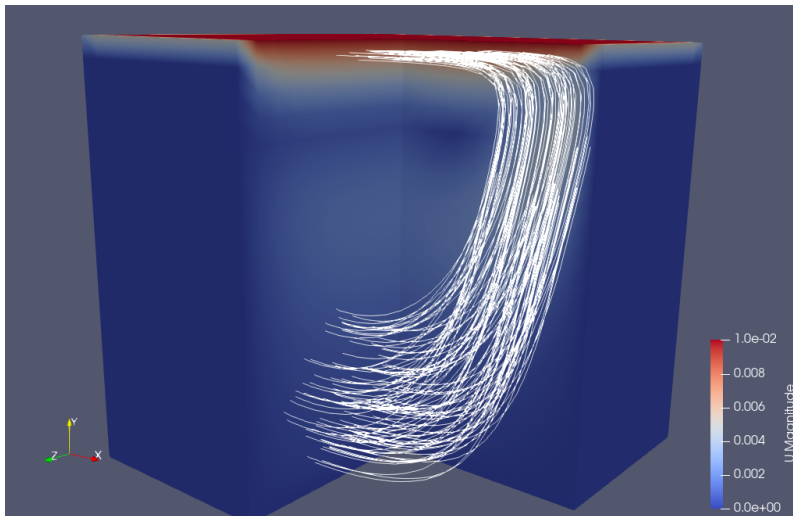


Table of Contents

- 1 Membrane Gas Separation
- 2 CFD
- 3 OpenFOAM
- 4 Simulation: Lid-Driven Cavity
- 5 Simulation: 3D Lid-Driven Cavity
- 6 Homework**
- 7 Conclusion
- 8 Supplementary Material
- 9 Papers

Homework: Lid-Driven Cavity Flow Study

Part 1: Reynolds Number Effects

- Simulate 2D cavity flow for $Re = 1, 100, \text{ and } 1000$
- Keep fixed: $L = 0.01 \text{ m}, \nu = 10^{-6} \text{ m}^2/\text{s}$
- Adjust top wall velocity U to achieve each Re
- Run until steady-state (monitor residuals!)

Part 2: Mesh Convergence Test

- For $Re = 100$, test 3 meshes:
 - Coarse: 10×10 cells
 - Medium: 20×20 cells (default)
 - Fine: 40×40 cells
- Compare centerline velocity profiles (x component of U and a function of y for $x = 0.005 \text{ m}$)
- Determine when results stop changing significantly

Deliverables:

- ParaView screenshots of:
 - Streamlines
 - Velocity magnitude contours
 - Pressure contours
- Brief report answering:
 - How flow changes from $Re = 1$ to $Re = 1000$?
 - Which mesh resolution gave stable results for $Re = 1000$?
 - What did you learn from the mesh test?

Tip: Use ParaView's *Plot Over Line* tool for quantitative comparisons.

Table of Contents

- 1 Membrane Gas Separation
- 2 CFD
- 3 OpenFOAM
- 4 Simulation: Lid-Driven Cavity
- 5 Simulation: 3D Lid-Driven Cavity
- 6 Homework
- 7 Conclusion**
- 8 Supplementary Material
- 9 Papers

Key topics from **Lecture 00**.

- Membrane separation fundamentals:
 - Governing equations
 - Boundary conditions
- Introduction to OpenFOAM:
 - Open-source advantages
 - Directory structure and workflow
- First simulations with OpenFOAM: the lid-driven cavity problem

Next Lecture:

- Mesh generation:
 - gmsh
 - snappyHexMesh
- Simulations:
 - Flow between parallel plates
 - Flow in a T-junction

Table of Contents

- 1 Membrane Gas Separation
- 2 CFD
- 3 OpenFOAM
- 4 Simulation: Lid-Driven Cavity
- 5 Simulation: 3D Lid-Driven Cavity
- 6 Homework
- 7 Conclusion
- 8 Supplementary Material**
- 9 Papers

0) **My complete course on OpenFOAM:** a detailed course on general CFD with OpenFOAM.

github.com/adrianopossebon/Curso-CFD-OpenFOAM-Introducao.

1) **LabCC/UFSC Introductory Course:** 6 lectures on youtube.

youtube.com/watch...

2) **3 Weeks Series:** to get a more detailed understanding of the basics of OpenFOAM.

wiki.openfoam.com

3) **Wolf dynamics:** tutorials and more material about OpenFOAM.
wolfdynamics.com/tutorials

4) Book: **The Finite Volume Method in Computational Fluid Dynamics: An Advanced Introduction with OpenFOAM and Matlab.** Moukalled, Mangani and Darwish.

5) Book: **Gas Separation Membranes: Polymeric and Inorganic.** Ismail, Khulbe and Matsuura.

6) OpenFOAM official site: openfoam.org

Table of Contents

- 1 Membrane Gas Separation
- 2 CFD
- 3 OpenFOAM
- 4 Simulation: Lid-Driven Cavity
- 5 Simulation: 3D Lid-Driven Cavity
- 6 Homework
- 7 Conclusion
- 8 Supplementary Material
- 9 Papers

A few reference papers for the project:

- **CFD Simulations:** [1, 2, 3, 4, 5, 6, 7, 8]
- **General Simulations:** [9, 10]
- **Review Papers:** [8, 11, 12, 13, 14]
- **Experimental Studies:** [15, 16, 17, 18, 19]

- [1] Bahram Haddadi, Christian Jordan, Martin Miltner, and Michael Harasek. Membrane modeling using CFD: Combined evaluation of mass transfer and geometrical influences in 1D and 3D. *Journal of Membrane Science*, 563(December 2017):199–209, 2018.
- [2] Nawaf Alkhamis, Ali E. Anqi, and Alparslan Oztekin. Computational study of gas separation using a hollow fiber membrane. *International Journal of Heat and Mass Transfer*, 89:749–759, 2015.
- [3] Nawaf Alkhamis, Dennis E. Oztekin, Ali E. Anqi, Abdulmohsen Alsaiani, and Alparslan Oztekin. Numerical study of gas separation using a membrane. *International Journal of Heat and Mass Transfer*, 80:835–843, 2015.
- [4] Fatemeh Banitaba, Zahra Mansourpour, and Shohreh Fatemi. Combination of Adsorption-Diffusion Model with Computational Fluid Dynamics for Simulation of a Tubular Membrane Made from SAPO-34 Thin Layer Supported by Stainless Steel for Separation of CO₂ from CH₄. *Journal of Membrane and Separation Technology*, 5(1):16–24, 2016.
- [5] Salman Qadir, Arshad Hussain, and Muhammad Ahsan. A computational fluid dynamics approach for the modeling of gas separation in membrane modules. *Processes*, 7(7), 2019.
- [6] Bahram Haddadi, Christian Jordan, Philipp Schretter, Tino Lassmann, and Michael Harasek. Designing Better Membrane Modules Using CFD. *Chemical Product and Process Modeling*, 11(1):57–66, 2016.
- [7] M Harasek, B Haddadi, M Miltner, P Schretter, and C Jordan. Fully resolved CFD simulation of a hollow fibre membrane module. *Chemical Engineering Transactions*, 52:433–438, 2016.
- [8] K. Foo, Y. Y. Liang, P. S. Goh, and D. F. Fletcher. Computational fluid dynamics simulations of membrane gas separation: Overview, challenges and future perspectives. *Chemical Engineering Research and Design*, 191:127–140, 2023.

- [9] Sina Gilassi, Seyed Mohammad Taghavi, Denis Rodrigue, and Serge Kaliaguine. Simulation of gas separation using partial element stage cut modeling of hollow fiber membrane modules. *AIChE Journal*, 64(5):1766–1777, 2018.
- [10] Shiguang Li, John L. Falconer, Richard D. Noble, and R. Krishna. Modeling permeation of CO₂/CH₄, CO₂/N₂, and N₂/CH₄ mixtures across SAPO-34 membrane with the Maxwell-Stefan equations. *Industrial and Engineering Chemistry Research*, 46(12):3904–3911, 2007.
- [11] Marcos Da Conceicao, Leo Nemetz, Joanna Rivero, Katherine Hornbostel, and Glenn Lipscomb. Gas Separation Membrane Module Modeling: A Comprehensive Review. *Membranes*, 13(7), 2023.
- [12] Alysson M A Silva, Anna Carolina, Britto De Faria, Camila De Lima, Daniel D Athayde, Edson Paulo, Giovana Magalh, Luiz Fernando, De Sousa Lima, Rodrigo Nunes, De Souza, Saulo Lucas, and Nelcy D S Mohallem. Review of zeolite membranes for natural gas processing and treatment : State of the art and future perspectives. *Gas Science and Engineering*, 117:205056, 2023.
- [13] Zishu Cao, Ninad D. Anjekar, and Shaowei Yang. Small-Pore Zeolite Membranes: A Review of Gas Separation Applications and Membrane Preparation. *Separations*, 9(47):1–27, 2022.
- [14] Moises A. Carreon. Molecular sieve membranes for N₂/CH₄ separation. *Journal of Materials Research*, 33(1):32–43, 2018.
- [15] Yanmei Li, Shengnan He, Chaojiu Shu, Xinping Li, Bo Liu, Rongfei Zhou, and Zhiping Lai. A facile approach to synthesize SSZ-13 membranes with ultrahigh N₂ permeances for efficient N₂/CH₄ separations. *Journal of Membrane Science*, 632(March):119349, 2021.
- [16] Shiguang Li, Zhaowang Zong, Shaojun James Zhou, Yi Huang, Zhuonan Song, Xuhui Feng, Rongfei Zhou, Howard S. Meyer, Miao Yu, and Moises A. Carreon. SAPO-34 Membranes for N₂/CH₄ separation: Preparation, characterization, separation performance and economic evaluation. *Journal of Membrane Science*, 487:141–151, 2015.

- [17] L. Tranchino, R. Santarossa, C. Fabiani, and L. Bimbi. Gas Separation in a Membrane Unit: Experimental Results and Theoretical Predictions. *Separation Science and Technology*, 24(14):1207–1226, 1989.
- [18] L. Yu, Mojtaba Sinaei Nobandegani, A. Holmgren, and J. Hedlund. Highly permeable and selective tubular zeolite CHA membranes. *Journal of Membrane Science*, 588(July):117224, 2019.
- [19] Yanghwan Jeong, Sejin Kim, Minseong Lee, Sungwon Hong, Mun Gi Jang, Nakwon Choi, Kyo Seon Hwang, Hionsuck Baik, Jin Kuk Kim, Alex C.K. Yip, and Jungkyu Choi. Supporting Information - A Hybrid Zeolite Membrane-Based Breakthrough for Simultaneous CO₂ Capture and CH₄ Upgrading from Biogas. *ACS Applied Materials and Interfaces*, 14(2):2893–2907, 2022.